

RabbitMQ

- 异步通讯，时效性较差、拿不到对方的结果，流量削峰填谷、接触耦合、无需等待、故障隔离如 RabbitMQ
- 同步通讯，时效性高，拓展性差、性能下降、级联失败，如openFeign

MQ技术选型：

	RabbitMQ	ActiveMQ	RocketMQ	Kafka
公司/社区	Rabbit	Apache	阿里	Apache
开发语言	Erlang	Java	Java	Scala&Java
协议支持	AMQP, XMPP, SMTP, STOMP	OpenWire, STOMP, REST, XMPP, AMQP	自定义协议	自定义协议
可用性	高	一般	高	高
单机吞吐量	一般	差	高	非常高
消息延迟	毫秒级	毫秒级	毫秒级	毫秒以内
消息可靠性	高	一般	高	一般

- Erlang:面向并发的语言

一、RabbitMQ简介

异步通讯

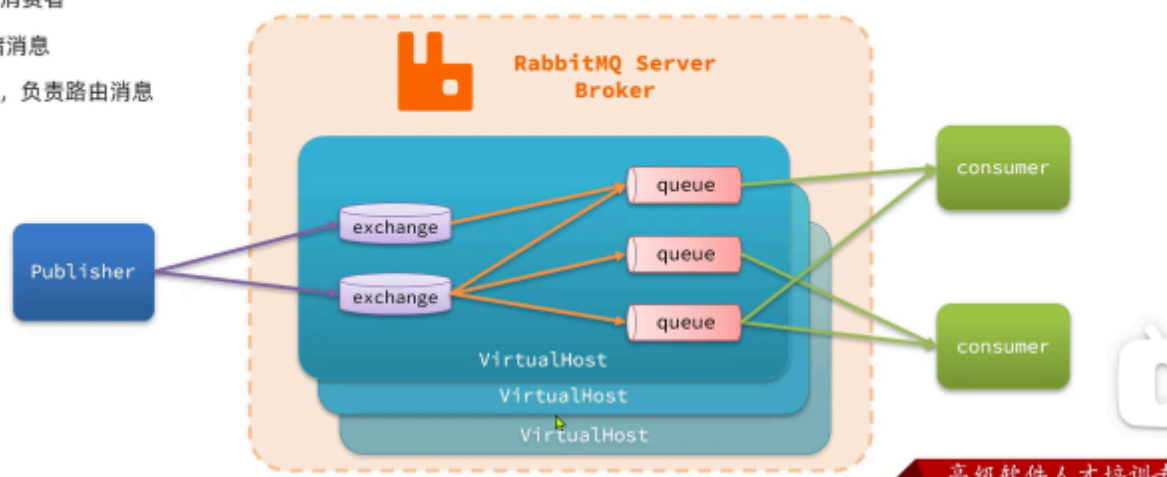
- 消息发送者：投递消息的人，就是原来的调用方
- 消息代理：管理、暂存、转发消息，你可以把它理解成微信服务器
- 消息接收者：接收和处理消息的人，就是原来的服务提供方



RabbitMQ

RabbitMQ的整体架构及核心概念：

- virtual-host: 虚拟主机，起到数据隔离的作用
- publisher: 消息发送者
- consumer: 消息的消费者
- queue: 队列，存储消息
- exchange: 交换机，负责路由消息



代码块

```
1  docker run \  
2  -e RABBITMQ_DEFAULT_USER=cyy \  
3  -e RABBITMQ_DEFAULT_PASS=123456 \  
4  -v mq-plugins:/plugins \  
5  --name mq \  
6  --hostname mq \  
7  -p 15672:15672 \  
8  -p 5672:5672 \  
9  --network cyy-rabbitMQ \  
10 -d \  
11 rabbitmq:3.8-management
```

注：自己可以填写USER/PASS/network

- 5672发消息
- 15672是控制台

二、快速入门

2.1 快速创建

1. 创建队列Q1、Q2
2. 向默认的交换机amp.fanout发送一条消息
3. 查看消息是否到达Q1、Q2
 - 没有到达，因为没有路由，交换机不存储消息

2.2 数据隔离

1. 创建新用户
 2. 创建新虚拟主机
- 为每个项目创建一个虚拟主机

三、Java客户端/SpringAMQP

AMQP是用于在应用程序之间传递业务消息的开放标准。该协议与语言和平台无关，更符合微服务中独立性的要求。

springAMQP:是一套统一的API，springamqp是基础抽象，spring-rabbit是底层的默认实现

3.1 依赖

代码块

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5          http://maven.apache.org/xsd/maven-4.0.0.xsd">
6
7      <groupId>com.cyy.demo</groupId>
8      <artifactId>mq-demo</artifactId>
9      <version>1.0-SNAPSHOT</version>
10     <modules>
11         <module>publisher</module>
12         <module>consumer</module>
13     </modules>
14     <packaging>pom</packaging>
15
16     <parent>
17         <groupId>org.springframework.boot</groupId>
18         <artifactId>spring-boot-starter-parent</artifactId>
19         <version>2.7.12</version>
20         <relativePath/>
21     </parent>
22
23     <properties>
24         <maven.compiler.source>8</maven.compiler.source>
25         <maven.compiler.target>8</maven.compiler.target>
26     </properties>
27
28     <dependencies>
29         <dependency>
30             <groupId>org.projectlombok</groupId>
```

```

31         <artifactId>lombok</artifactId>
32     </dependency>
33     <!--AMQP依赖, 包含RabbitMQ-->
34     <dependency>
35         <groupId>org.springframework.boot</groupId>
36         <artifactId>spring-boot-starter-amqp</artifactId>
37     </dependency>
38     <!--单元测试-->
39     <dependency>
40         <groupId>org.springframework.boot</groupId>
41         <artifactId>spring-boot-starter-test</artifactId>
42     </dependency>
43 </dependencies>
44 </project>

```

3.2 快速入门

Publicher - queue - consumer

1. 利用控制台创建队列simple.queue
2. 在publiclser服务中, 利用SpringAMQP直接向simple.queue发送消息
 - 配置mq服务端信息

代码块

```

1  spring:
2      rabbitmq:
3          host: 192.168.100.102
4          port: 5672
5          virtual-host: /rabbitmq-learn
6          password: 123456
7          username: rabbitmq-learn

```

- RabbitTemplate工具类

代码块

```

1  @Autowired
2  private RabbitTemplate rabbitTemplate;
3
4  @Test
5  public void testSendMessage2Queue(){
6      String queueName = "simple.queue";
7      String message = "hello";
8      rabbitTemplate.convertAndSend(queueName,message);

```

3. 在consumer服务中，利用SpringAMQP编写消费者，监听simple.queue队列

- 可以通过@RabbitListener的queues注解来指定监听的队列

代码块

```

1  @Slf4j
2  @Component
3  public class MqListener {
4      @RabbitListener(queues = "simple.queue")
5      public void getMqMessage(String msg){//String取决于发送的类型
6          System.out.println("消费者收到了消息：【"+msg+"】");
7      }
8  }

```

3.3 work模型

让多个消费者绑定到一个队列，共同消费队列中的信息

步骤：

1. 创建work.queue
2. 在publisher服务中定义测试方法，1秒产生50条消息，发送
3. 在consumer中定义两个监听者监听work
4. 消费者1每秒处理50条消息，消费者2每秒处理5条消息

结论：

- 正常队列中一条消息只能被一个消费者消费
- 一般是轮询机制，一个消费者消费一次
- 哪怕目前只有某个消费者在监听，它依然会只拿自己的部分（轮询）

修改：

代码块

```

1  spring:
2      rabbitmq:
3          listener:
4              simple:
5                  prefetch: 1# 给消费者加上，每次只能处理一条消息，处理完成才能获取下一条

```

3.4 fanout/direct/topic交换机

真的生产环境都会经过exchange来发送消息，而不是直接发送到队列，交换机类型有以下三种：

1. fanout:广播

代码块

```
1  @Test
2  public void testSendMessageToFanout(){
3      String exchangeName = "mq.fanout";
4      String message = "hello, consumer";
5      rabbitTemplate.convertAndSend(exchangeName,null,message); //可以不传
6  }
```

2. direct:定向

- 一个交换机和一个队列可指定多个Bindingkey
- 发布者发送消息的时候，指定routingkey
- 可以广播（只要routingkey和bindingkey一样）

代码块

```
1  @Test
2  public void testSendMessageToFanout(){
3      String exchangeName = "mq.fanout";
4      String message = "hello, consumer";
5      rabbitTemplate.convertAndSend(exchangeName,"routingkey",message); //可以不传
6  }
```

3. topic:话题

- 与direct类似，但是routingkey可以是多个单词的列表，并且以“.”分割
- queue和exchange指定Bindingkey时可以使用通配符：
 - #代指0个或多个单词
 - *代指一个单词

3.5 声明队列交换机[java代码]

手动添加会很麻烦，而且不可复刻

方法：

1. Queue/Exchange/Binding类用来声明，都可以用xxxBuilder来创建【一般在消费者那里声明】

代码块

```

1  @Configuration
2  public class FanoutConfig {
3      @Bean
4      public FanoutExchange fanoutExchange(){
5          return new FanoutExchange("xxx.fanout");//也可以用Builder
6          fanoutExchange
7      }
8      @Bean
9      public Queue fanoutQueue(){
10         return new Queue("xxx.queue");//也可以用Builder durable
11     }
12     @Bean
13     public Binding bindingQueue(Queue fanoutQueue, FanoutExchange
14     fanoutExchange){
15         return BindingBuilder.bind(fanoutQueue).to(fanoutExchange);
16     }
17 }

```

缺点：对于direct，若要绑定多个routingKey需要很多个Bean

2. 注解声明

代码块

```

1  @RabbitListener(bindings = @QueueBinding(
2      value = @Queue(name="xxx.queue",durable="true"),
3      exchange = @Exchange(name = "xxx.xxx",type= ExchangeTypes.DIRECT),
4      key={"1","2"}
5  ))

```

3.6 消息转换器

案例：

1. 声明一个队列，名为object.queue
2. 编写单元测试，向队列中直接发送一条消息，消息类型为Map
3. 再在控制台查看消息，总结
 - 用的是jdk序列法
 - 转完后体积很大
 - 可能有安全风险

修改：

- a. 采用json序列法，需要引入jackson依赖

```

代码块
1 <dependency>
2     <groupId>com.fasterxml.jackson.core</groupId>
3     <artifactId>jackson-databind</artifactId>
4 </dependency>

```

b. 配置MessageConverter

```

代码块
1 @Bean
2 public MessageConverter messageConverter(){
3     return new Jackson2JsonMessageConverter();
4 }

```

四、MQ高级

解决：消息可靠性问题（保证消息至少被消费一次）

4.1 生产者可靠性

4.1.1 生产者重连

由于网络波动，可能会出现客户端连接MQ失败的情况，通过配置开启连接失败后的重连机制

```

代码块
1 spring:
2     rabbitmq:
3         template:
4             retry:
5                 enabled: true #开启超时重试机制
6                 initial-interval: 1000ms #失败后的初始等待时间
7                 multiplier: 1 #失败后的下次等待时长倍数，下次等待时长=initial-interval *
multiplier
8                 max-attempts: 3 #最大重试次数

```

1. 提供的是阻塞式的重试
2. 有性能要求建议禁用重试机制
3. 可以考虑用异步线程来之星发送消息的代码

4.1.2 生产者确认

RabbitMQ有两种确认机制：Publisher Confirm和Publisher Return。开启后成功收到消息后会返回确认消息给生产者。

情况：

1. 消息投递到了MQ，但是路由失败。此时会通过Return返回路由异常原因，然后返回ACK，告知投递成功
2. 临时消息投递到了MQ，并且入队成功，返回ACK，告知投递成功
3. 持久消息投递到了MQ，入队并且完成吃就会，返回ACK，告知投递成功
4. 其它都会返回NACK，告知投递失败

步骤：

1. 依赖

代码块

```
1  #publisher
2  spring:
3      rabbitmq:
4          publisher-confirm-type: correlated #none关闭 simple同步阻塞等待
          correlated异步回调
5          publisher-returns: true #路由失败返回，一般情况下不需要开启
```

2. 设置ReturnCallback

注意：每个RabbitTemplate只能配置一个ReturnCallback，需要在启动项目配置

代码块

```
1  @Slf4j
2  @Configuration
3  public class CommonConfig implements ApplicationContextAware { //当容器初始化完
    成，就可以传给该类使用
4      @Override
5      public void setApplicationContext(ApplicationContext applicationContext)
        throws BeansException {
6          //获取RabbitTemplate
7          RabbitTemplate rabbitTemplate =
        applicationContext.getBean(RabbitTemplate.class);
8          //设置ReturnCallback
9          rabbitTemplate.setReturnsCallback(returned -> {
10              log.info("ReturnCallback: message={}, replyCode={}, replyText={},
                exchange={}, routingKey={}",
11                  returned.getMessage().toString(),
12                  returned.getReplyCode(),
13                  returned.getReplyText(),
14                  returned.getExchange(),
15                  returned.getRoutingKey());
16          });
```

```
17     }
18 }
```

3. ConfirmCallback

- 在每个消息发送的时候单独指定

代码块

```
1  @Test
2  void testPublisherConfirm(){
3      //1. 创建correlationData (该对话的唯一ID, uuid)
4      CorrelationData correlationData = new CorrelationData();
5      //2. 给Future添加ConfirmCallback
6      correlationData.getFuture().addCallback(new
7      ListenableFutureCallback<CorrelationData.Confirm>() {
8          @Override
9          public void onFailure(Throwable ex) {
10             //Future发生异常时的处理逻辑
11             log.error("错误");
12         }
13         @Override
14         public void onSuccess(@Nullable CorrelationData.Confirm result)
15         { //spring内部失败
16             if(result.isAck()) {
17                 log.info("消息发送成功");
18             } else {
19                 log.info("消息发送失败");//result.getReason()String类型,异常描述
20             }
21         }
22     });
23     //3. 发送消息
24     rabbitTemplate.convertAndSend("confirm.exchange","confirm.queue",
25     "hello,confirm", correlationData);
26 }
```

4.2 MQ的可靠性

4.2.1 数据持久化

MQ默认将消息保存在内存中，问题：

1. MQ宕机，数据丢失
2. 内存空间有限，消息积压，引发MQ阻塞

解决：

1. 交换机和队列的持久化：交换机和队列都选Durable，不选Transient
 - 注：在spring中创建默认都是持久化的
2. 消息的持久化：发送的时候Properties的delivery_mode改为2
 - 注：在spring中发送默认是持久化的

4.2.2 Lazy Queue

特征：

1. 接收到消息后**直接**存入磁盘而非内存（内存中只保存最近的消息，默认2048条）
2. 消费者要消费消息时才会从磁盘中读取并加载到内存
3. 支持数百万条消息存储
 - 在3.12版本后，所有队列都是Lazy Queue,无法更改

设置：x-queue-mode = lazy（现在默认就是）

代码块

```
1 public Queue lazyQueue(){
2     return QueueBuilder.durable("name").lazy().build();
3 }
```

代码块

```
1 @RabbitListener(queuesToDeclare = @Queue(
2     name = "lazy.queue",
3     durable = "true",
4     arguments = @Argument(name = "x-queue-mode",value = "lazy"))//添加
5 ))
```

4.3 消费者的可靠性

4.3.1 消费者确认机制

消息：

1. Ack：队列删除
2. Nack：队列重发
3. reject:消息本身有问题，拒接，队列删除

Spring amqp已经实现了消息确认功能，用户可以通过配置文件选择三种ACK处理方式：

1. None:不处理。只返回ACK。

2. manual:手动模式，需要自己在业务代码中调用api，发送ack或者reject,存在业务入侵，但是更灵活
3. auto：自动模式。环绕增强，业务正常执行时自动返回ACK，业务出现异常时，若业务异常nack，若消息处理或者校验异常reject

代码块

```
1  spring:
2    rabbitmq:
3      listener:
4        simple:
5          acknowledge-mode: none #none manual auto
```

4.3.2 消费失败处理策略

问题：不断入队发送带来很大压力

解决：用spring的retry机制，在消费者出现异常时利用本地重试，而不是无限制的重新入队到新队列

代码块

```
1  spring:
2    rabbitmq:
3      listener:
4        simple:
5          retry:
6            enabled: true # 消费者失败重试
7            initial-interval: 1000ms #初始失败等待时长
8            multiplier: 1 #下次失败的等待时长倍数，下次等待市场 = m * l
9            max-attempts: 3 #最大重试次数
10           stateless: true # true无状态，false有状态。如果业务中包含事务，这里为false
```

- 多次重试失败会把消息扔掉

解决：用MessageRecover接口处理，包含三种实现：

1. RejectAndDontRequeueRecoverer:重试耗尽后直接reject丢弃，默认
2. ImmediateRequeueMessageRecoverer:重试耗尽后，返回nack，消息入队
3. RepublishMessageRecoverer:重试耗尽后，将失败消息投递到指定的交换机【可以设置一个专门接受失败消息的交换机】

第三种方案：

1. 定义交换机和队列
2. 定义RepublishMessageRecoverer

代码块

```
1 public MessageRecoverer republishMessageRecoverer(RabbitTemplate
    rabbitTemplate){
2     return new
    RepublishMessageRecoverer(rabbitTemplate,"error.direct","errorKey");
3 }
4 }
```

- @ConditionalOnProperty()//指定当yml开启时才生效ErrorConfig

4.3.3 业务幂等性

方案:

1. 唯一id

代码块

```
1 @Configuration
2 public class MessageConverter {
3     @Bean
4     public Jackson2JsonMessageConverter messageConverter(){
5         //1. 定义消息转换器
6         Jackson2JsonMessageConverter jjmc = new Jackson2JsonMessageConverter();
7         //2. 配置自动创建消息id, 用于识别不同消息, 也可以在业务中基于ID判断是否重复消息
8         jjmc.setCreateMessageIds(true);
9         return jjmc;
10    }
11 }
```

问题: 1. 业务方有额外操作

2. 基于业务本身做判断

举例: 只有未支付状态的订单才需要修改

4.4 延迟消息

生产者发送的时候指定一个时间, 消费者不会立刻收到消息, 而是在指定时间之后才收到消息

举例: 未支付订单

4.4.1 死信交换机

死信 (dead letter):

1. 消费者使用basic.reject或basic.nack声明消费失败, 并且设置消息的requeue参数设置为false
2. 消息是一个过期消息 (达到了队列或消息本身设置的过期时间), 超时无消费
3. 要投递的队列消息堆积满了, 最早的消息可能称为死信

如果队列通过dead-letter-exchange属性指定了也给交换机那么该队列中的死信就会投递到这个交换机中。这个交换机称为死信交换机（DLX）

- 添加过期时间setExpiration

代码块

```
1  Message message =  
    MessageBuilder.withBody("hello").getBytes(StandardCharsets.UTF_8)).setExpiration  
    ("10000").build();  
2  //10秒，毫秒的单位  
3  //上述不能用消息转换器，否则会报错
```

4.4.2 延迟消息插件

Rabbitmq推出的插件，原生支持延迟消息功能。原理是设计了一种支持延迟消息功能的交换机，当消息投递到交换机后可以暂存一定时间，到期后再投递到队列

- delayed_message_exchange，可以在github下载

步骤：

1. 查看存放路径

代码块

```
1  docker inspect mq
```

找到：name:plugins的source

2. 复制github的包到上述位置

3. 执行命令

代码块

```
1  docker exec -it mq rabbitmq-plugins enable rabbitmq_delayed_message_exchange
```

使用：

1.

代码块

```
1  //注解  
2  @RabbitListener(bindings = @QueueBinding(  
3      value = @Queue(name="xxx.queue",durable="true"),
```

```
4         exchange = @Exchange(name = "xxx.xxx", type=
      ExchangeTypes.DIRECT, delayed="true"), //添加delayed
5         key={"1", "2"}
6     ))
7     //bean方式直接用delayed()
```

2.

代码块

```
1     message.getMessageProperties().setDelay(5000)
```

问题：会有性能损耗

4.4.3 取消超时订单

问题：

1. 大多数1分钟就支付了，但是订单需要再MQ等待30分钟，浪费资源
2. MQ积压30分钟的内容压力大

解决：把长的延迟消息拆分成一个个短的延迟消息

- 自定义一个MultiDelayMessage，有data和List<Long>delayMillis

4.4.4 监听延迟消息